5.1 Objectives

- To introduce the reader to Spring MVC framework
- To know about the features of the Spring MVC framework
- To understand the process to configure applications using frameworks with illustrations

Spring framework has integration support with other ORM, MVC and other frameworks. Spring allows for an easy integration with Struts MVC framework with web applications could be built. Spring also provide it own MVC framework to build web applications.

5.2 Course Content

5.2.1 Introduction

Spring framework has integration support with other ORM, MVC and other frameworks. Spring allows for an easy integration with Struts MVC framework with web applications could be built. Spring also provide it own MVC framework to build web applications.

5.2.2 Understanding MVC Flow

Model-View-Controller architecture is used for interactive web-applications. This model minimizes the coupling between business logic and data presentation to web user. This model divides the web based application into three layers:

- **Model**: Model domain contains the business logics and functions that manipulate the business data. It provides updated information to view domain and also gives response to query. And the controller can access the functionality which is encapsulated in the model.

- **View**: View is responsible for presentation aspect of application according to the model data and also responsible to forward query response to the controller.
- **Controller**: Controller accepts and intercepts user requests and controls the business objects to fulfill these requests. An application has one controller for related functionality. Controller can also be depends on the type of clients.

5.2.3 Introduction to Spring MVC

The Spring Web model-view-controller (MVC) framework is designed around a DispatcherServlet that dispatches requests to handlers, with configurable handler mappings, view resolution, locale, time zone and theme resolution as well as support for uploading

5.2.3.1 Data Binding

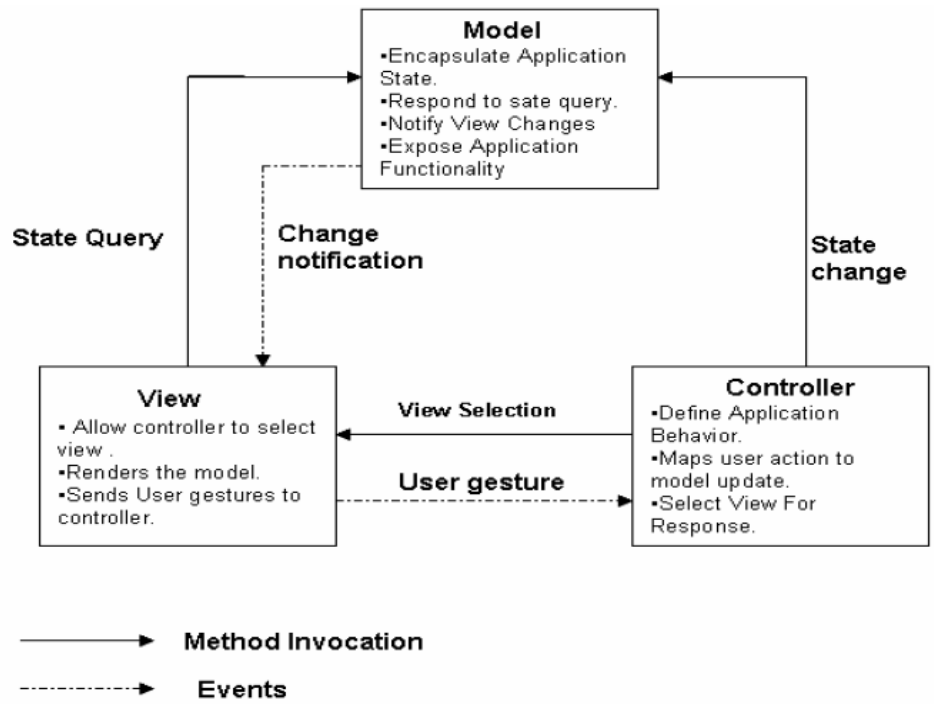


Figure 1: MVC – Data Binding

In Spring Web MVC you can use any object as a command or form-backing object; you do not need to implement a framework-specific interface or base class.

Spring's data binding is highly flexible: for example, it treats type mismatches as validation errors that can be evaluated by the application, not as system errors. Thus you need not duplicate your business objects' properties as simple, untyped strings in your form objects simply to handle invalid submissions, or to convert the Strings properly. Instead, it is often preferable to bind directly to your business objects.

5.2.3.2 View Resolution

Spring's view resolution is extremely flexible. A Controller is typically responsible for preparing a model Map with data and selecting a view name but it can also write directly to the response stream and complete the request. View name resolution is highly configurable through file extension or accept header content type negotiation, through bean names, a properties file, or even a custom ViewResolver implementation.

5.2.3.3 Model Map

The Model (the M in MVC) is a Map interface, which allows for the complete abstraction of the view technology. You can integrate directly with template based rendering technologies such as JSP, Velocity and Freemarker, or directly generate XML, JSON, Atom, and many other types of content. The model Map is simply transformed into an appropriate format, such as JSP request attributes, a Velocity template model.

5.2.3.4 Features of Sring MVC

The following are some of the features of spring MVC framework

- Clear separation of roles. Each role — controller, validator, command object, form object, model object, DispatcherServlet, handler mapping, view resolver, and so on can be fulfilled by a specialized object.
- Powerful and straightforward configuration of both framework and application classes as JavaBeans. This configuration capability includes easy referencing across contexts, such as from web controllers to business objects and validators.

- Adaptability, non-intrusiveness, and flexibility. Define any controller method signature you need, possibly using one of the parameter annotations (such as @RequestParam, @RequestHeader, @PathVariable, and more) for a given scenario.
- Reusable business code, no need for duplication. Use existing business objects as command or form objects instead of mirroring them to extend a particular framework base class.

5.2.3.5 Front Controller Design Pattern

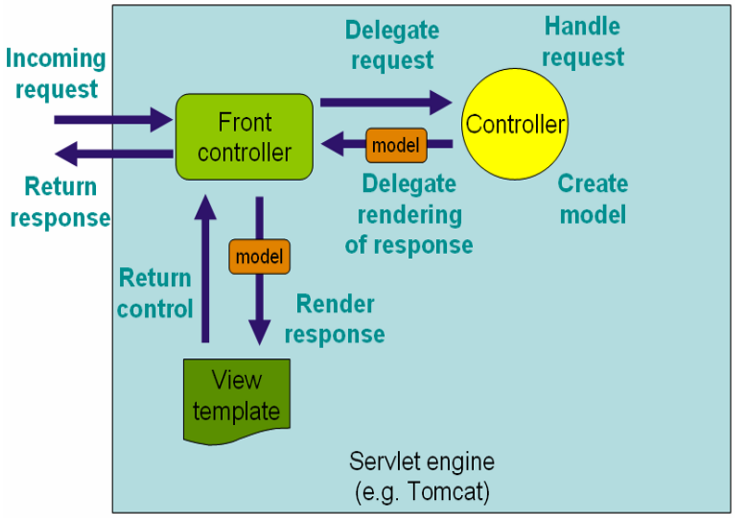
The front controller design pattern is used to provide a centralized request handling mechanism so that all requests will be handled by a single handler. This handler can do the authentication/ authorization/ logging or tracking of request and then pass the requests to corresponding handlers. Following are the entities of this type of design pattern.

- **Front Controller** - Single handler for all kind of request coming to the application (either web based/ desktop based).
- **Dispatcher** - Front Controller may use a dispatcher object which can dispatch the request to corresponding specific handler.
- **View** - Views are the object for which the requests are made

5.2.3.6 Dispatcher Servlet

Spring's web MVC framework is, like many other web MVC frameworks, request-driven, designed around a central servlet that dispatches requests to controllers and offers other functionality facilitating the development of web applications. Spring's DispatcherServlet however, does more than just that. It is completely integrated with the Spring IoC container and as such allows you to use every other feature that Spring has.

The request processing workflow of the Spring Web MVC DispatcherServlet is illustrated in the following diagram. The DispatcherServlet is an expression of the “Front Controller” design pattern.



5.2.4 Spring MVC Configuration

5.2.4.1 Configuring Dispatcher Sevlet

The DispatcherServlet is an actual Servlet (it inherits from the HttpServlet base class), and as such is declared in the web.xml of your web application. Requests that you want the DispatcherServlet to handle will have to be mapped using a URL mapping in the same web.xml file. This is standard J2EE servlet configuration; an example of such a DispatcherServlet declaration and mapping can be found below.

```
<web-app>
  <servlet>
    <servlet-name>example</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet- class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>example</servlet-name>
    <url-pattern>*.form</url-pattern>
  </servlet-mapping>
</web-app>
```

The framework will, on initialization of a DispatcherServlet, look for a file named [servlet-name]-servlet.xml in the WEB-INF directory of your web application and create the beans defined there. With the above servlet configuration in place, you will need to

have a file called '/WEB-INF/example-servlet.xml' in your application; this file will contain all of your Spring Web MVC-specific components (beans). The exact location of this configuration file can be changed via a servlet initialization parameter.

In a Servlet 3.0+ environment, you also have the option of configuring the Servlet container programmatically. Below is the code based equivalent of the above web.xml example:

```
public class MyWebApplicationInitializer implements WebApplicationInitializer {
    @Override
    public void onStartup(ServletContext container) {
        ServletRegistration.Dynamic registration = container.addServlet("dispatcher", new DispatcherServlet());
        registration.setLoadOnStartup(1);
        registration.addMapping("/example/*");
    }
}
```

WebApplicationInitializer is an interface provided by Spring MVC that ensures your code-based configuration is detected and automatically used to initialize any Servlet 3 container

5.2.4.2 Configuring Web Application Context

5.2.4.2.1 Spring MVC Themes

The theme support provided by the Spring web MVC framework enables you to further enhance the user experience by allowing the look and feel of your application to be themed. A theme is basically a collection of static resources affecting the visual style of the application, typically style sheets and images.

5.2.4.2.2 Spring Application Context

Application Context includes all functionality of the Bean Factory. While the beans package provides basic functionality for managing and manipulating beans, often

in a programmatic way, the context package adds Application Context, which enhances Bean Factory functionality in a more framework-oriented style.

5.2.4.2.3 Spring Web Application Context

ApplicationContext instances in Spring can be scoped. In the web MVC framework, each DispatcherServlet has its own WebApplicationContext, which inherits all the beans already defined in the root WebApplicationContext. These inherited beans defined can be overridden in the servlet-specific scope, and new scope-specific beans can be defined local to a given servlet instance.

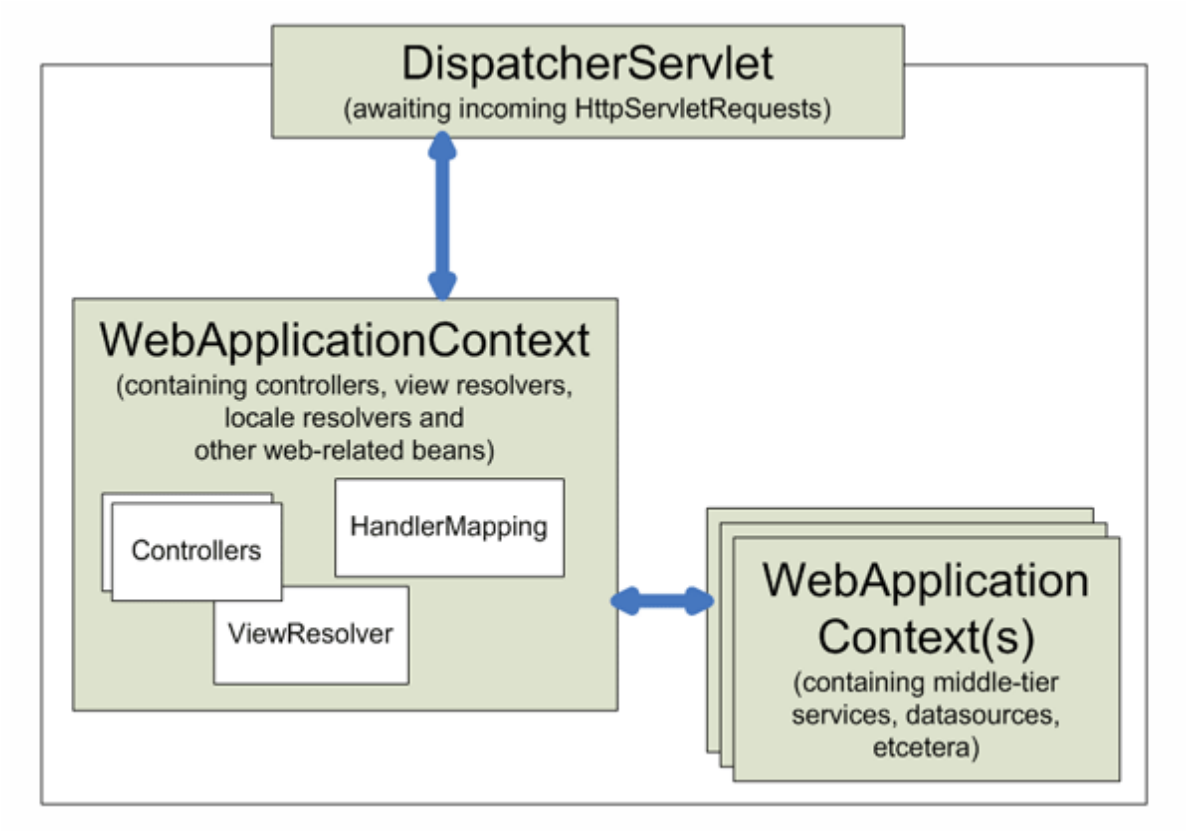


Figure 2: Spring WebApplicationContext

The WebApplicationContext is an extension of the plain ApplicationContext that has some extra features necessary for web applications. It differs from a normal

ApplicationContext in that it is capable of resolving themes. and that it knows which servlet it is associated with (by having a link to the ServletContext). The WebApplicationContext is bound in the ServletContext.

You can customize Spring's DispatcherServlet by adding context parameters in the web.xml file or servlet initialization parameters. The possibilities are listed below.

- contextClass Class - that implements WebApplicationContext, which will be used to instantiate the context used by this servlet. If this parameter isn't specified, the XmlWebApplicationContext will be used.
- contextConfigLocation String - which is passed to the context instance (specified by contextClass) to indicate where context(s) can be found. The string is potentially split up into multiple strings (using a comma as a delimiter) to support multiple contexts (in case of multiple context locations, of beans that are defined twice, the latest takes precedence).
- namespace - the namespace of the WebApplicationContext. Defaults to [servlet-name]-servlet.

```
<web-app>
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/root-context.xml</param-value>
  </context-param>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <init-param>
      <param-name>contextConfigLocation</param-name>
      <param-value></param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
  </servlet>
```

```
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>/*</url-pattern>
  </servlet-mapping>
  <listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
  </listener>
</web-app>
```

ApplicationContext is where your Spring beans live. The purpose of the ContextLoaderListener is two-fold:

1. To tie the lifecycle of the ApplicationContext to the lifecycle of the ServletContext and
2. To automate the creation of the ApplicationContext, so you don't have to write explicit code to do create it - it's a convenience function.

Another convenient thing about the ServletContextListener is that it creates a WebApplicationContext and WebApplicationContext provides access to the ServletContext via ServletContextAware beans and the getServletContext method.ContextLoaderListener is optional.

It is wise sometimes to split all the Configuration information across multiple Configuration Files. In such a case we have to depend on a Listener Servlet called Context Loader represented by org.springframework.web.context.ContextLoaderListener.

```
<web-app>
<listener>
<listener-class>
org.springframework.web.context.ContextLoaderListener
</listener-class>
```

```
</listener>

</web-app>
```

By default, this Context Listener will try to look for the Configuration File by name 'applicationContext.xml' in the '/WEB-INF' directory. But with the help of the parameter 'contextConfigLocation' the default location can be overridden. Even multiple Configuration Files each containing separate piece of Information is also possible.

web.xml

```
<?xml version='1.0' encoding='UTF-8'?>

<web-app version='2.4'>

<listener>

<listener-class>

org.springframework.web.context.ContextLoaderListener

</listener-class>

</listener>

<context-param>

<param-name>contextConfigLocation</param-name>

<param-value>/WEB-INF/contacts.xml, /WEB-INF/resources.xml</param-value>

</context-param>

</web-app>
```

The above definition instructs the Framework to look and load for the Configuration Files by name 'contacts.xml' and 'resources.xml' in the WEB-INF directory.

5.2.4.2.4 Special Beans in Web Application Context

The Spring DispatcherServlet has a couple of special beans it uses in order to be able to process requests and render the appropriate views. These beans are included in

the Spring framework and can be configured in the WebApplicationContext, just as any other bean would be configured.

- **HandlerMapping:** Maps incoming requests to handlers and a list of pre- and post-processors (handler interceptors) based on some criteria the details of which vary by HandlerMapping implementation. The most popular implementation supports annotated controllers but other implementations exists as well.
- **HandlerAdapter:** Helps the DispatcherServlet to invoke a handler mapped to a request regardless of the handler is actually invoked. For example, invoking an annotated controller requires resolving various annotations. Thus the main purpose of a HandlerAdapter is to shield the DispatcherServlet from such details.
- **HandlerExceptionResolver:** Maps exceptions to views also allowing for more complex exception handling code.
- **ViewResolver:** Resolves logical String-based view names to actual View types.
- **LocaleResolver & LocaleContextResolver:** Resolves the locale a client is using and possibly their time zone, in order to be able to offer internationalized views
- **ThemeResolver:** Resolves themes your web application can use, for example, to offer personalized layouts
- **MultipartResolver:** Parses multi-part requests for example to support *processing* file uploads from HTML forms.
- **FlashMapManager:** Stores and retrieves the "input" and the "output" FlashMap that can be used to pass attributes from one request to another, usually across a redirect.

5.2.4.3 Handler Mapping

When the Client Request reaches the Dispatcher Servlet, the Dispatcher Servlet tries to find the appropriate Handler Mapping Object to map between the Request and the Handling Object. A Handler Mapping provides an abstract way that tell how the Client's Url has to be mapped to the Handlers. Four concrete variation of Handler Mapping are available. They are defined as follows

- BeanNameUrlHandlerMapping
- CommonsPathMapHandlerMapping
- ControllerClassNameHandlerMapping
- SimpleUrlHandlerMapping

In plain MVC using Servlets and JSP it is done through servlet mapping in web.xml file. Check the code snippet below.

```
<?xml version="1.0" encoding="ISO-8859-1" ?>

<web-app xmlns="http://java.sun.com/xml/ns/j2ee"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd"

version="2.4">

<display-name>HelloWorld Application</display-name>

<description>

This is a simple web application with a source code organization

based on the recommendations of the Application Developer's Guide.

</description>

<servlet>

<servlet-name>HelloServlet</servlet-name>

<servlet-class>examples.Hello</servlet-class>

</servlet>
```

```
<servlet-mapping>

<servlet-name>HelloServlet</servlet-name>

<url-pattern>/hello</url-pattern>

</servlet-mapping>

</web-app>
```

In Spring MVC, URL mapping is done through handlers. Handlers will be discussed in details in the coming sections.

5.2.4.3.1 Bean NameUrlHandlerMapping

The BeanNameUrlHandlerMapping class maps URL requests to beans names. It is the default handler mapping class, so it is the one created by the DispatcherServlet when Spring cannot find any handler mapping class declared. An example of using the BeanNameUrlHandlerMapping class is shown below.There is a beans declared, and is named as CreateContact.htm and its class is the CreateContactController. So the BeanNameUrlHandlerMapping will map any Create Contact URL request to this Controller.

```
<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE beans PUBLIC '-//SPRING//DTD BEAN//EN'
'http://www.springframework.org/dtd/spring-beans.dtd'>
<beans>
<bean id='beanNameUrlMapping' class='org.springframework.web.servlet.handler.
BeanNameUrlHandlerMapping'/>


<bean name = '/CreateContact.htm' class='net.javabeat.articles.spring.mvc.contacts.CreateContactController'>
<property name='formView'>
<value>CreateContact</value>
</property>
<property name='successView'>
<value>ContactCreated</value>
</property>
</bean>
```

```
</beans>
```

The controller class can be called from the jsp file using the URL Mapping as given in the example below.

```
<html>
<head>
<meta http-equiv='Content-Type' content='text/html; charset=UTF-8'>
<title>Create a Contact</title>
</head>
<body>
<h1>Create a Contact</h1>
<form name = 'CreateContact' method = 'get">
<input type = 'text' name = 'firstname' />
<input type = 'text' name = 'lastname' />
<br>
<input type='submit' name = 'Create Contact' value = 'Create Contact'/>
</form>
</body>
</html>
```

5.2.4.3.2 CommonsPathMapHandlerMapping

The CommonsPathMapHandlerMapping provide the facility to the name of URL which are mapped to the controller source file for example if we are to map 'loginPage','registerationPage' and 'welcomePage' to Controllers name LoginController,RegisterationController and WelcomeController,then the mapping information allocate in meta-data in the source files inside the Javadoc comments.

This handler mapping example which are following:-

```
<beans>
        <bean id="metaHandlerMapping" class="org.springframework.web.servlet.handler.
        metadata.CommonsPathMapHandlerMapping"/>
</beans>
```

5.2.4.3.3 ControllerClassNameHandlerMapping

This handler provide the facility to handle the more then one Handler Mapping, the Controller name using directly from the Url itself with some modifications.

This handler mapping are show in the following:-

```
<beans ...>
        <bean class="org.springframework.web.servlet.mvc. support.ControllerClassNameHandlerMapping"
        />
        <bean class="r4r.co.controller.WelcomeController" />
        <bean class="r4r.co.controller.HelloGuestController" />
</beans>
```

5.2.4.3.4 SimpleUrlHandlerMapping

This handler provides the facility to handle more than one handler map directly to the Client Request to some Controller object. Example of this handler which are following:

```
<beans ...>
        <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
                <property name="mappings">
                        <props>
                                <prop key="/welcome.bean">welcomeController</prop>
                                <prop key="/login.bean">loginController</prop>
                                <prop key="/registeration.bean">registerationController</prop>
                                </props>
                </property>
        </bean>
        <bean id="welcomeController" class="com.controller.WelcomeController" />
        <bean id="loginController" class="com.controller.LoginController" />
        <bean id="RegisterationController" class="com.controller.RegisterationController" />
</beans>
```

5.2.5 Spring Controllers

5.2.5.1 Spring Controller Actions

Spring MVC module is based on the MVC design pattern. The main components involved are DispatcherServlet, Controller and Views. In Spring MVC DispatcherServlet plays very important role. It handles the user request and delegates it with Controller. The **Controller** is responsible for processing user requests and building appropriate model and passes it to the view for rendering.

Following is the sequence of events corresponding to an incoming HTTP request to DispatcherServlet:

- After receiving an HTTP request, DispatcherServlet consults the HandlerMapping to call the appropriate Controller.
- The Controller takes the request and calls the appropriate service methods based on used GET or POST method. The service method will set model data based on defined business logic and returns view name to the DispatcherServlet.
- The DispatcherServlet will take help from ViewResolver to pick up the defined view for the request.
- Once view is finalized, The DispatcherServlet passes the model data to the view which is finally rendered on the browser.

This can be observed in the figure shown below:

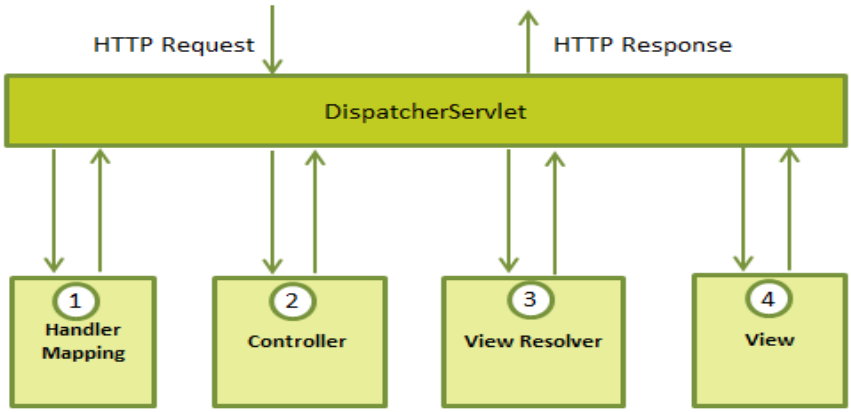


Figure 3: Process by a Dispatcher Servlet

5.2.5.2 Controller Hierarchy

Spring MVC provides many abstract controllers, which is designed for specific tasks. Here is the list of abstract controllers that comes with the Spring MVC module:

- SimpleFormController
- AbstractController
- AbstractCommandController
- CancellableFormController
- AbstractCommandController
- MultiActionController
- ParameterizableViewController
- ServletForwardingController
- ServletWrappingController
- UrlFilenameViewController
- AbstractController
- AbstractCommandController

- SimpleFormController
- CancellableFormController

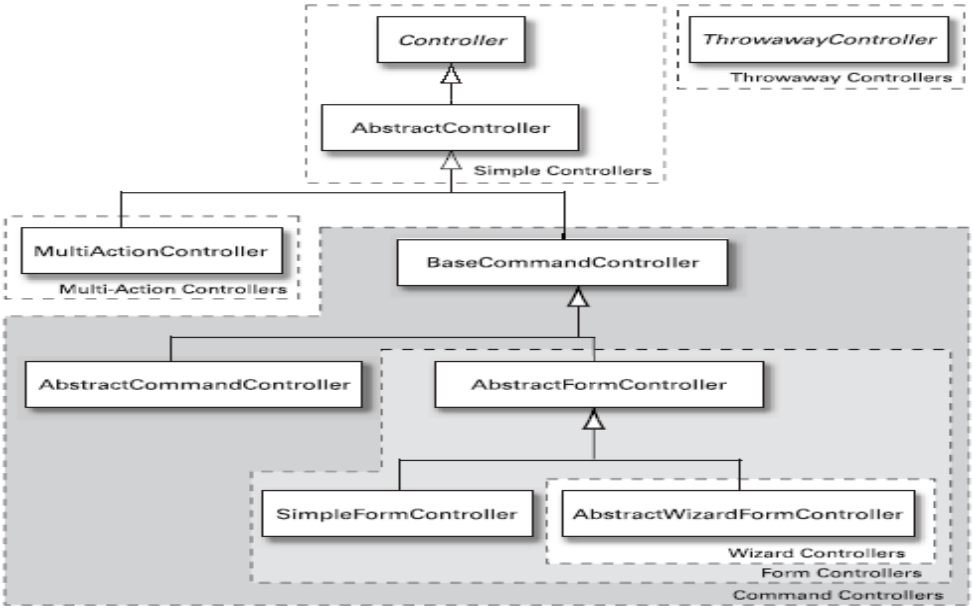


Figure 4: Controllers Hierarchy

5.2.6 View Resolver

In Spring MVC, view resolvers enable you to render models in a browser without tying you to a specific view technology like JSP, Velocity, XML…etc.There are two interfaces that are important to the way Spring handles views are ViewResolver and View. The ViewResolver provides a mapping between view names and actual views. The View interface addresses the preparation of the request and hands the request over to one of the view technologies.

Below are the important viewresolvers provided by spring framework:

- **AbstractCachingViewResolver :** Abstract view resolver that caches views. Often views need preparation before they can be used; extending this view resolver provides caching.

- **XmlViewResolver :** Implementation of ViewResolver that accepts a configuration file written in XML with the same DTD as Spring's XML bean factories. The default configuration file is /WEB-INF/views.xml.
- **ResourceBundleViewResolver :** Implementation of ViewResolver that uses bean definitions in a ResourceBundle, specified by the bundle base name. Typically you define the bundle in a properties file, located in the classpath. The default file name is views.properties.
- **UrlBasedViewResolver :** Simple implementation of the ViewResolver interface that effects the direct resolution of logical view names to URLs, without an explicit mapping definition. This is appropriate if your logical names match the names of your view resources in a straightforward manner, without the need for arbitrary mappings.
- **InternalResourceViewResolver :** Convenient subclass of UrlBasedViewResolver that supports InternalResourceView (in effect, Servlets and JSPs) and subclasses such as JstlView and TilesView. You can specify the view class for all views generated by this resolver by using setViewClass(..).
- **VelocityViewResolver/FreeMarkerViewResolver:** Convenient subclass of UrlBasedViewResolver that supports VelocityView (in effect, Velocity templates) or FreeMarkerView ,respectively, and custom subclasses of them.
- **ContentNegotiatingViewResolver :** Implementation of the ViewResolver interface that resolves a view based on the request file name or Accept header.

Refer the below url for more details

http://candidjava.com/spring-mvc-with-order-view-resolvers-tutorial-and-example

5.2.7 Spring MVC Example

Step 1: Create a index page – index.jsp

```
<a href="simpleUrlOne.bean">First User Request</a><br/>
```

```
<a href="simpleUrlTwo.bean">Second User Request</a>
```

Step 2: Configure web.xml

```
<web-app id="WebApp_ID" version="2.4"
xmlns="http://java.sun.com/xml/ns/j2ee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">

	<display-name>Spring MVC Application</display-name>
	<servlet>
		<servlet-name>dispatcher</servlet-name>
		<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
		<load-on-startup>1</load-on-startup>
	</servlet>
	<servlet-mapping>
		<servlet-name>dispatcher</servlet-name>
		<url-pattern>*.bean</url-pattern>
	</servlet-mapping>
	<context-param>
		<param-name>contextConfigLocation</param-name>
		<param-value>/WEB-INF/dispatcher-servlet.xml</param-value>
	</context-param>
	<listener>
		<listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
	</listener>
	<welcome-file-list>
		<welcome-file>index.jsp</welcome-file>
	</welcome-file-list>
</web-app>
```

Step 3: Configure Spring beans (dispatcher-servlet.xml)

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.5.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-2.5.xsd">
	<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
		<property name="mappings">
			<props>
				<prop key="/simpleUrlOne.bean">firstController</prop>
				<prop key="/simpleUrlTwo.bean">secondController</prop>
			</props>
		</property>
	</bean>
	<bean id="firstController" class="com.test.FirstController" />
	<bean id="secondController" class="com.test.SecondController" />
	<bean
	id="viewResolver"class="org.springframework.web.servlet.view.InternalResourceViewResolver" >
		<property name="prefix">
			<value>/WEB-INF/pages/</value>
		</property>
		<property name="suffix">
			<value>.jsp</value>
		</property>
	</bean>
</beans>
```

Step 4: Create the first Controller Class

```
package com.test;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
public class FirstController extends AbstractController {
	public ModelAndView handleRequestInternal(HttpServletRequest request, HttpServletResponse
	response) {
		ModelAndView model = new ModelAndView("page");
		model.addObject("msg", "This message is generated by first controller!");
		return model;
```

```
        }
}
```

Step 5: Create the second Controller Class

```
package com.test;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
public class SecondController extends AbstractController {
        public ModelAndView handleRequestInternal(HttpServletRequest request, HttpServletResponse
        response) {
                ModelAndView model = new ModelAndView("page");
                model.addObject("msg", "This message is generated by second controller!");
                return model;
        }
}
```

Step 6: Create UI page

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<html>
        <head>
                <title>SimpleUrlHandler</title>
        </head>
        <body>
                <h3>SimpleUrlHandler Example in Spring MVC</h3>
                <H4>${msg}</H4>
        </body>
</html>
```

5.2.8 References

http://docs.spring.io/spring/docs/current/spring-framework-reference/html/mvc.html

http://docs.spring.io/spring-framework/docs/2.0.8/reference/mvc.html

http://krishnasblog.com/tag/spring-mvc/

http://candidjava.com/spring-mvc-with-order-view-resolvers-tutorial-and-example

http://examples.javacodegeeks.com/enterprise-java/spring/mvc/spring-mvc-handler-mapping-example/

http://www.roseindia.net/spring/spring-mvc-hello-world.shtml

http://r4r.co.in/java/spring/basic/example/springMVC_ex/Example_of_SimpleUrlHandlerMapping.shtml